

ECS7012P Coursework: Granulation System

GEWENXIN YU

Queen Mary University London

Email: `ec22214@qmul.ac.uk`

Granular synthesis and additive synthesis share the concept of creating complex sounds from basic elements. The granular synthesis system developed in the Bela platform allows users to manipulate audio samples by dividing them into small grains and reassembling them in real-time to create new sounds. The system will provide users with several parameters that can be adjusted in a GUI, including grain size, grain density, grain start position and grain spreading, allowing for a wide range of creative possibilities.

1. INTRODUCTION

Granular synthesis or granulation is a technique that allows for the transformation of sound through the manipulation of small sound particles [4].

Sergio Cavaliere and Aldo Piccialli [4] classified granular synthesis into two basic forms in terms of the method used to distribute grains in time, *synchronous granular synthesis* and *asynchronous granular synthesis*. *Synchronous granular synthesis* is a filtering technique that repeats the grains at fixed intervals and then weights the grains, which is the result of the convolution of a periodic pulse train with the grain, used as the impulse response of a system. However, in an *asynchronous granular synthesis*, the pulses are not emitted regularly as an excitation function. This method distributes grains according to a stochastic function that is traditionally specified in terms of grain density.

Dennis Gabor [2] was the first to introduce the notion of *sound quanta* into acoustic theory, drawing on quantum mechanics. Gabor's representation consisted of sinusoids with a Gaussian envelope window. The center frequency of the grain is controlled by the frequency of the sine wave, and the grain duration is determined by the width of the Gaussian envelope [4]. As described, time and frequency play symmetrical parts in this new approach to analyzing signals in Gabor's grain theory [2]. Grains are packets of waves distributed in both the time and frequency domains.

In addition, the Greek composer Iannis Xenakis is known as the first to explicate a compositional theory for *sound grains* [5]. As he described: "All sound is an integration of grains, of elementary sonic particles, of sonic quanta. Each of these elementary grains has a threshold nature: duration, frequency, and intensity". [7] These assertions set the stage for the development of more sophisticated computer music and real-time synthesis, and the gradual introduction of this theory into

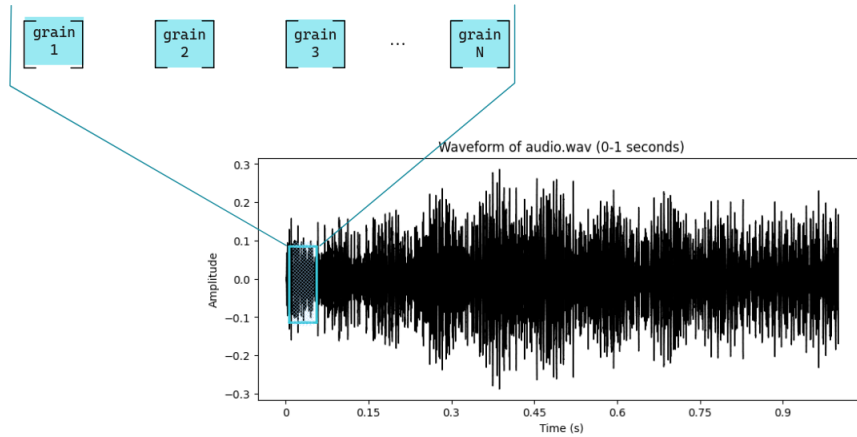


Figure 1. An audio is granulated and the first N grains are selected for the looping. The samples in the figure are the `audio.wav` used in this project, and the waveform is drawn from 0 to 1 second, using the `librosa.display.waveshow` function. The block size (audio frames) is 16 by default in this system, which means that every 16 samples will be formed into one block for the real-time computation.

electronic music and other musical genres. MUSICV was subsequently utilised by Curtis Road to realise Xenakis’s concept, whilst Barry Truax succeeded in creating a real-time granular synthesis system, GSX and GSAMX [1][6].

Granular synthesis represents a distinct perspective on sound, music, and composition [3]. This theory was a milestone in the field of computer music, opening up more possibilities for the exploration of sound.

The basic grain waveform in a regular granular synthesis system can be sinusoidal, sampled, or derived by deconvolution. *The grains can also be derived from natural sounds or from instrumental models* [4]. This work implements the basic idea of *synchronous granular synthesis*, using a sample file as the source file and splitting it into multiple grains of equal length to achieve the granulation effect. Figure 1. shows the basic idea of *synchronous granular synthesis* using a sample file as input.

The paper is organised as follows: Section 2 outlines the architecture and design of the granulation system, covering the correlation between grain size and distance within an audio file (illustrated in a figure), the method of grain indexing for fixed length, the algorithmic design of the granulation effect output, and the design of the Biquad low-pass filter. The subjective evaluation results are explored in Section 3, and possible areas for further research and improvement are discussed in Section 4.

2. SYSTEM OVERVIEW

The current system offers basic granular synthesis functionalities such as the option to modify grain size and grain distance (grain density). The Bela project comprises of `render.cpp`, `MonoFilePlayer.h` and `MonoFilePlayer.cpp`. A video demo is available as supplemental material.

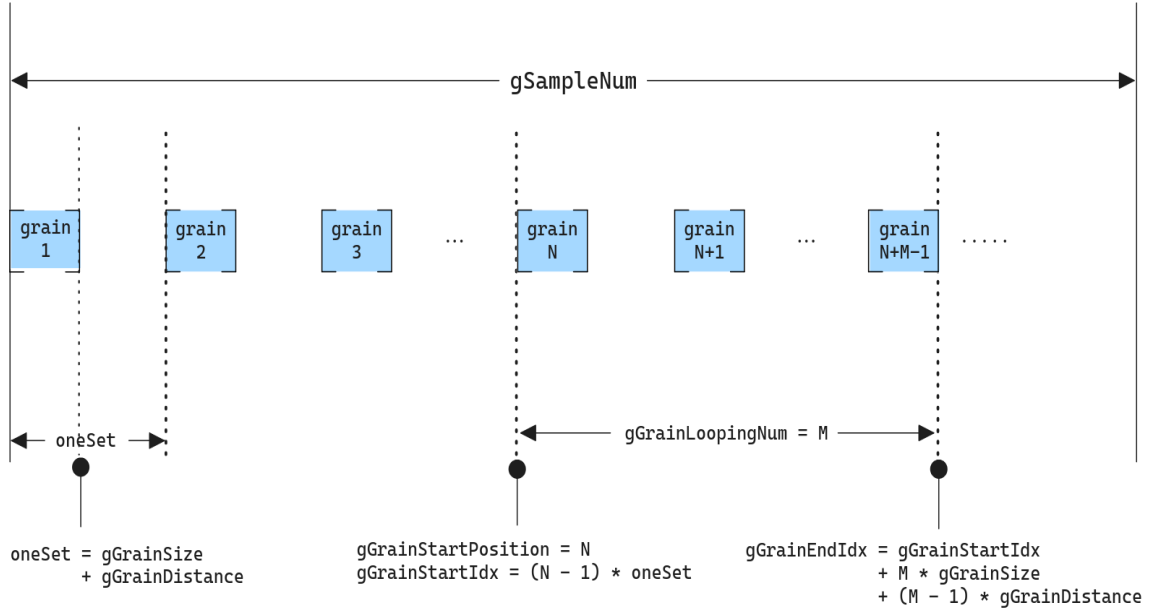


Figure 2. A graphical presentation of the calculation of the relationship between several important variables: $gGrainStartIdx$, $gGrainEndIdx$. The top bracket refers to an entire loaded audio file, in this case `audio.wav`, and $gSampleNum$ refers to the total number of samples in the audio file. The blue boxes visually represent each grain segmented before the output (not the actual segmentation of the source file). `grain1` represents the first grain of the audio file, similarly, `grainN` represents the N_{th} grain, and the interval from grain N to grain $N + M - 1$ represents the segment that the user intends to loop. The number of samples occupied by a grain plus the number of samples occupied by the (fixed) distance between grains is represented overall as `oneSet`.

2.1. Main algorithms

2.1.1. Method of designing output

In a real-time system, operations such as splitting, rewinding and partial playback of audio files are executed using read pointers instead of physically splitting the source file. This enables speedy reads and optimal utilization of computer resources.

First, the system reads the audio file at a rate of one block per 16 samples by specifying `gPlayer.process()` in a block-by-block loop. The `process()` function is used to return the next sample of the loaded audio file, all of which are declared in the `MonoFilePlayer.cpp`. And it specifies that `readPointer_` (the pointer defined to read the `sampleBuffer_` that holds all the samples) should start indexing at `grainStartIdx` and end at `grainEndIdx`. The main computational idea of `grainStartIdx` and `grainEndIdx` is shown in Figure 2.

The first output value is assigned the return value of `gPlayer.process()`. Subsequently, `out` undergoes low pass filtering.

The `outPointer` is declared as the primary pointer for writing to the audio output channel. If the `outPointer` is inside the grain region (i.e., smaller than the grain size), iterate through all audio output channels using a simple for loop to write `out` to them. If the `outPointer` is outside the grain region (i.e., smaller than one grain set but larger than the grain size), set a decay factor and multiply it by `out`

before writing to all the channels. At the end of each `if` statement, the `outPointer` must be accumulated. Reset the pointer to zero if it is beyond the entire grain set.

2.1.2. The decay factor F_{decay}

The decay factor F_{decay} , which ranges from 0 to 1.0, is used to attenuate the audio signal when it's outside the grain region (non-blue region). This attenuation creates a natural fading effect for the sound. The decay factor is calculated based on the position of the current sample relative to the end of a grain. The decay factor F_{decay} is calculated as

$$F_{\text{decay}} = 1 - \frac{N_{\text{outPointer}} - N_{\text{grainSize}}}{N_{\text{grainDistance}}} \quad (1)$$

where $N_{\text{outPointer}}$ is the value of `outPointer`, $N_{\text{grainSize}}$ is the number of grain size in samples, and $N_{\text{grainDistance}}$ is the number of grain distance in samples. By computing $\frac{N_{\text{outPointer}} - N_{\text{grainSize}}}{N_{\text{grainDistance}}}$, a value is obtained that represents the proportion of the distance from the sample position within the grain to the end of the grain. Subtracting this value from 1 yields the decay factor. The decay factor gradually decreasing as the sample position moves from inside the grain toward the end of the grain, thus achieving the desired audio fading effect. This approach to attenuation ensures a natural sound decay, avoiding abrupt clipping artifacts and smoothly transitioning the grain to silence.

2.1.3. `MonoFilePlayer::setGrainStartAndEndIdx()`

This function is used to send the value of the calculated `gGrainStartIdx` and `gGrainEndIdx` to the function `MonoFilePlayer::process()`. This allows for adjusting both the position and size of the loop region. These two values are used to indicate the index of the loop region, from which sample it starts and to which sample it ends.

Figure 2. illustrates how these two values can be calculated from some known values. When a loop is started from the N_{th} grain, the number of all samples before this will be the value of `gGrainStartIdx`, which is the number of samples corresponding to the $N - 1$ grain sets.

The value of `gGrainEndIdx`, on the other hand, is added on top of `gGrainStartIdx` to the total number of samples occupied by the distances between the M grains and $M - 1$ grains in the loop region.

2.2. Biquad low-pass filter

Butterworth filters are known for its characteristics within the maximum flat magnitude, which is as flat as possible within the passband, and are therefore resistant to spikes and glitches in interfering signals. The Q value is set to 0.707, and the cutoff frequency F_c is set to 1.7 kHz.

The transfer function of a biquad filter is the ratio of two quadratic functions in the z domain:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{a_0 + a_1 z^{-1} + a_2 z^{-2}} \quad (2)$$

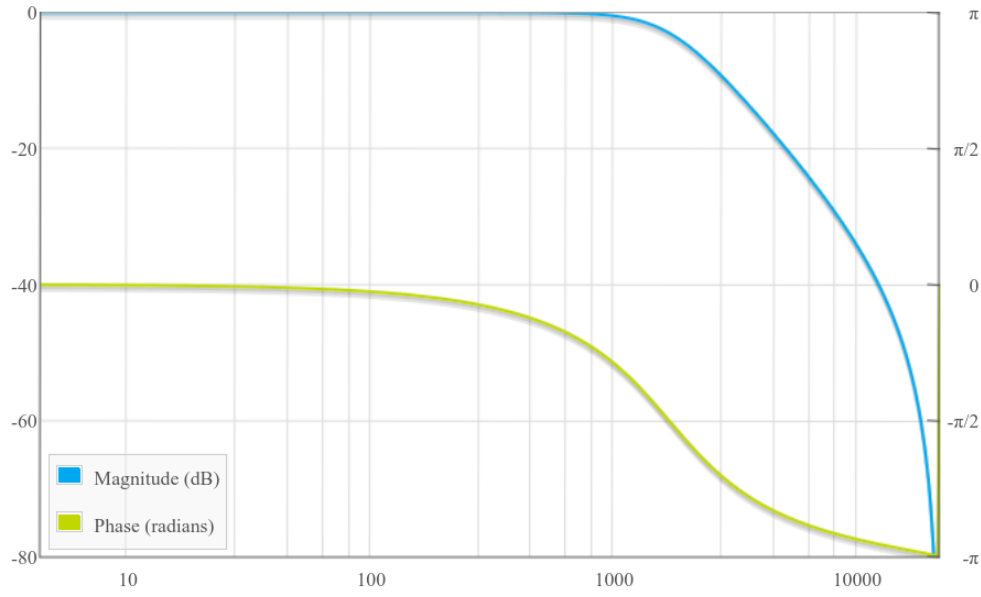


Figure 3. The frequency response of this low-pass filter, plotted logarithmically. $F_c = 1700$ Hz, $Q = 0.707$ and the sampling rate is 44.1 kHz. The magnitude response of this filter slopes off towards negative infinity.

where $a_0 = 0.012478147224361465$, $a_1 = 0.02495629444872293$, $a_2 = 0.012478147224361465$, $b_0 = 1$, $b_1 = -1.660039312267674$, $b_2 = 0.7099519011651199$. This is automatically calculated by the biquad calculator from Nigel Redmon ¹, also the frequency response of this low-pass Butterworth filter is shown in Figure 3.

In this project, the use of the biquad low-pass filter serves to minimise the effect of abrupt clipping artifacts. In addition, it also brings an effect of *telephone filter*.

2.3. Design of parameters

The GUI design of this system is shown in Figure 4. The following subsections describe how these four parameters are designed and how they change the effect of granulation.

Controls		
Grain Size	<input type="range"/>	1000
Grain Density	<input type="range"/>	2000
Grain Start Position	<input type="range"/>	1
Grain Spreading	<input type="range"/>	5

Figure 4. The Graphical User Interface (GUI) design of this system allows for adjustable Grain Size, ranging from 40 to 5000. Similarly, Grain Density can be adjusted between 0.2 and 1. Additionally, users can alter Grain Start Position, which ranges from 1 to 100, and Grain Spreading, ranging from 1 to 20.

1. <https://www.earlevel.com/main/2021/09/02/biquad-calculator-v3/>

2.3.1. Grain size

As described by Sergio Cavaliere and Aldo Piccialli [4], musical practice has used a wide range of grain durations, ranging from 1ms to 100ms. When the sampling rate is 44.1 kHz, the corresponding samples range from 44.1 to 4410. For a smoother effect, the range of values is set to 40 to 5000, with a step size of 40.

2.3.2. Grain density

Grain density is in this case the inverse of grain distance, and it is also mapped to a range from 0 to 1.0. The minimum value of grain density is set to 0.2 with a step size of 0.008.

2.3.3. Grain start position

Grain start position is the start position of the loop region and it is counted in grains rather than samples.

2.3.4. Grain spreading

Grain spread is a more intuitive way of describing the number of grains in a loop.

3. EVALUATION

This project successfully demonstrates what a *synchronous granular synthesis* can accomplish by allowing the user to freely start a loop at many possible positions in a sample, and then adjust the length of this loop region, as well as adjusting to the appropriate grain size and grain density.

All possible clipping sounds are handled by a Butterworth low-pass filter and a decay factor, a special effect is demonstrated by this low-pass filter.

There are some weaknesses in this project, the first of which is that the grain size and grain distance in the loop or in the `sampleBuffer` is always not divisible. A more efficient and accurate method of calculation should be implemented to solve the clipping problem.

Also, as the four parameters are not dynamically linked, the static slider value may not function effectively on some very short audio files. In fact, the four parameters are closely linked, and their precise ranges can be defined, with each parameter's range potentially changing with any modification to the others, resulting in an improved user experience.

4. FUTURE DEVELOPMENT

Future development involves implementing random indexing in this system to enable randomisation in the loop region. In addition, the transposition effect, as demonstrated in the Multi-Touch HTML5 Granular Synthesizer 1.0², can utilize the transposition effect within a granular synthesis system to produce a more captivating result.

2. HTML5 Granular Synthesiser 1.0: <https://github.com/zya/granular>

BIBLIOGRAPHY

- [1] Giovanni De Poli, Aldo Piccialli, and Curtis Roads, editors. *Representations of Musical Signals*. MIT Press, Cambridge, MA, USA, 1991.
- [2] Dennis Gabor. Acoustical quanta and the theory of hearing. *Nature*, 159:591–594, 1947.
- [3] Toru Iwatake and Barry Truax. An interview with barry truax. *Computer Music Journal*, 18:17, 1994.
- [4] Curtis Roads, Aldo Piccialli, Giovanni De Poli, and Stephen T. Pope. *Musical Signal Processing*. Swets & Zeitlinger, USA, 1997.
- [5] Curtis Roads and John Strawn, editors. *Foundations of Computer Music*. Massachusetts Institute of Technology, USA, 1985.
- [6] Barry Truax. Real-time granular synthesis with a digital signal processor. *Computer Music Journal*, 12:14, 1988.
- [7] Iannis Xenakis. *Formalized Music: Thought and Mathematics in Composition*. Bloomington: Indiana University Press, 1971.